MyCodeSchool

Code review — vicentiu-recapitulare-initiere-cpp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 5 — 2026-10-23 — sha 129eeb4

Am compilat fiecare header separat și am rulat funcțiile pe exemplele din fișă, plus capetele. Logica de la munte/vale și de la ștergere e în mare parte bună; ce te oprește acum sunt câteva detalii de ordine și de tip.

Poarta de compilare: `main.cpp` compilează, dar doar pentru că include numai `stergere.h`. Separat: `algoritmi.h`, `minim_maxim.h`, `sortare_bule.h`, `sortare_selectie.h`, `sortare-interval.h`, `stergere.h` trec; `cautare_binara.h` (3 erori), `interclasare.h` și `munte_vale.h` pică (B28). Deci 🔴 pică, vezi B28.

Fișa, secțiunile 6–8:

✅ corecte	6.1–6.5 (logica, verificată pe o copie reparată) · 7.2 (`1 2 4 7 9`) · 8.1 · 8.5
🔴 rezultat greșit	8.2 · 8.3 · 8.4
⏳ nescrise	7.1 (fără `sol`) · 7.3 · 7.4 · 7.5

Închise din rundele trecute: nimic nou în runda asta; vezi starea fiecărei constatări în rundele de mai jos.

🔴 Critice

B28 🔴 `munte_vale.h` și `interclasare.h` nu compilează când le incluzi `munte_vale.h:8` · `interclasare.h:72`

CUM E ACUM: `munte_vale.h:8–9` și `:14–15` cheamă `sortareInterval` / `sortareIntervalDescrescator`, definite abia la `:88` și `:106`; `sortareMunte` e definită de două ori (`:6` și `:125`). `interclasare.h:72` și `:86` folosesc `MAX`, care nu e definit nicăieri.

```
munte_vale.h:8:5: error: use of undeclared identifier 'sortareInterval'
munte_vale.h:125:6: error: redefinition of 'sortareMunte'      (5 erori)
interclasare.h:72:11: error: use of undeclared identifier 'MAX'
interclasare.h:74, :77, :79: use of undeclared identifier 'y'
```

DE CE: Compilatorul citește fișierul o singură dată, de sus în jos. La linia 8 încă n-a văzut `sortareInterval`, deci pentru el numele nu există, chiar dacă apare 80 de linii mai jos. Același nume de funcție definit de două ori înseamnă două corpuri pentru același apel, iar compilatorul nu are cum să aleagă. În `interclasare.h`, `int x[MAX], y[MAX];` pică din cauza lui `MAX`, așa că nici `y` nu se mai declară: cele 3 erori cu `y` vin tot de acolo, nu sunt bug-uri separate. De ce F9 era verde: `main.cpp` include doar `stergere.h`, iar în `.cbp` headerele au target `None`, deci compilatorul nu le-a citit niciodată.

Logica e corectă: pe o copie reparată în scratchpad, 6.1–6.5 dau exact rezultatele din fișă, iar 7.2 dă `1 2 4 7 9`.

FIX: funcțiile de bază sus, înaintea celor care le cheamă; `sortareMunte` o singură dată; în loc de `MAX`, o dimensiune scrisă de tine (`int x[100]`). Apoi fiecare header, pe rând, în `main.cpp` și F9.

- același bug la `munte_vale.h:8`, `:9`, `:14`, `:15`, `:125`, `:88`, `:106` · `interclasare.h:72`, `:74`, `:77`, `:79`, `:86`, `:87`, `:90` · `cautare_binara.h:203`, `:206`, `:195` · `algoritmi.h:140`, `:159`, `:178` · `sortare-interval.h:40`, `:59` · `stergere.h:5`, `:41`, `:70` · `interclasare.h:6`, `:81` · `parcurgerea-vectoriilor.h:66`
- lipsește `#include <iostream>` în `parcurgerea-vectoriilor.h:1` și `sortate_bule.h:1`

💡 Fiecare funcție e definită înainte de prima ei folosire? Verifică fiecare header, pe rând, singur în `main.cpp`.

B29 🔴 8.2: `stergerePare` sare peste parul de după un par șters `stergere.h:65`

CUM E ACUM: după `stergereElement` lipsește `i--`.

```
sol2()                  ->  14 15 10 14 84    trebuia 15
stergerePare {5,8,2,9}  ->  5 2 9             trebuia 5 9
```

DE CE: `stergereElement` mută toate elementele de după `poz` cu o poziție la stânga, deci cel care era pe `i+1` ajunge pe `i`. Apoi `for`-ul face `i++` și trece la `i+1`, așa că elementul tocmai mutat pe `i` nu mai e verificat niciodată. În `10 14 10 ...`, `14` alunecă peste `10`, iar bucla a trecut deja de locul ăla. Pe exemplul din fișă (`5 8 3 2 9`) iese bine pentru că parele nu sunt vecine. Regula ai aplicat-o corect în `sol1:51`, `sol3:112` și `sol4:139`, doar aici lipsește.

FIX: `i--` după `stergereElement(v, d, i)`, ca la `sol1` (sau parcurgi de la coadă spre început).

- același bug: nu mai există alt loc cu ștergere în buclă fără `i--`. Celelalte apeluri (`stergere.h:50`, `:111`, `:138`) au `i--`, iar `algoritmi.h:550` e un apel unic, fără buclă. Singurul loc de reparat e `stergere.h:65`.
- defecte vecine, altă regulă: `stergere.h:109` (B30), `:126` (B31), `:131–132` (C24).

💡 Pe `10 14 10 15`: ce se află pe poziția 1 imediat după ce ștergi `14` de pe poziția 1, și ce poziție verifică `for`-ul la pasul următor?

B30 🔴 8.3: sortarea strică ordinea, iar ultimul element e șters de propria fantomă `stergere.h:106` · `:109`

CUM E ACUM: `sol3` sortează vectorul înainte de ștergere, deci nu mai păstrează prima apariție în ordinea inițială. În plus, la `i = d-1`, `v[i+1]` citește dincolo de capăt.

```
{5,2,5,7,2,2,9}, d=7  ->  2 5 7        trebuia 5 2 7 9
sol3()                ->  10 12 14 15 83 84     (lipsește 92)
```

DE CE: `d--` doar spune că vectorul e mai scurt, nu șterge efectiv căsuța `v[d]`. După o ștergere, ultimul element a fost copiat cu o poziție la stânga, dar valoarea lui a rămas și pe vechiul loc. Când `i` ajunge la ultimul element, `v[i+1]` e chiar căsuța aceea, cu aceeași valoare, și duplicatul e găsit acolo; de aceea `9` dispare. ASan nu se plânge pentru că `v` are 100 de căsuțe, deci citirea e în memorie validă, dar e în afara celor `d` elemente. Sortarea e a doua problemă: fișa cere prima apariție, iar după sortare nu mai știi care a fost prima.

FIX: fără sortare: pentru fiecare `i`, ștergi toate `j > i` cu `v[j] == v[i]` (cu `j--` după ștergere). Bucla merge doar pe `i < d`, nu compara cu `v[i+1]` la ultimul.

- același bug la `stergere.h:109` (citire `v[i+1]` la `i = d-1`, cauză separată de sortare; ambele trebuie reparate)
- vezi și `stergere.h:65` (B29) și `:132` (C24): alte probleme în aceeași zonă, altă regulă
- verificat fără încălcări: sortările din `algoritmi.h:85,104,148,167`, `munte_vale.h:96,114`, `sortare-interval.h:48,68`, `sortare_bule.h:95` și `stergere.h:91` merg până la `d-1`; `munte_vale.h:145` are `i < n - 1`; `stergere.h:173` (sol5) folosește `d=11` corect; duplicatul `stergereElement` din `algoritmi.h:227` e corect.

💡 Pe `5 2 5 7 2 2 9`: ce e în `v[d-1]` și ce e în `v[d]` înainte de ultima comparație?

B31 🔴 8.4: media aritmetică calculată cu împărțire întreagă `stergere.h:126` · `:133`

CUM E ACUM: `return s / d;` cu `s` și `d` de tip `int`, plus `int ma = ...` la `:133`.

```
mediaAritmetica({1,2}, 2)  ->  1
sol4 pe {1,2}              ->  1 2      trebuia 2
```

DE CE: Tipul unei împărțiri îl dau operanzii, nu variabila în care pui rezultatul. `int / int` face împărțire întreagă: `3 / 2 = 1`, iar abia apoi `1` e transformat în `1.0`, la `return`. Faptul că funcția întoarce `double` nu salvează nimic, pentru că restul s-a pierdut deja. Apoi `int ma` taie încă o dată orice zecimală. Pe exemplul din fișă (`4 8 1 7`, media 5) iese bine doar pentru că media e număr întreg.

FIX: cel puțin un operand real: `(double)s / d`, iar `double ma` în `sol4`.

- același bug la `stergere.h:133`: `int ma = mediaAritmetica(v, d);` trunchiază a doua oară; fără `double ma`, reparația de la `:126` nu are efect
- același bug la `sortare_bule.h:12`: `return (v[dim / 2 - 1] + v[dim / 2]) / 2;` în `mediana()`; enunțul (`:6`) cere media celor două din mijloc. Pe `{35,56,19,42}`, după sortare mijlocul e `35` și `42`: iese `38` în loc de `38.5`. Trebuie tip de întoarcere `double` și `/ 2.0`; `sol31` (`:25`) afișează rezultatul
- verificat și neafectate (nu sunt medii): cifre `/ 10`, mijloc binar `/ 2` pe indici (`algoritmi.h:59`, `cautare_binara.h:15/144/177`), `dim / 2` și `n / 3` ca indici (`sortare-interval.h:81-135`), `aux2 / aux` la cmmmc (`while-ex.h:346`, împărțire exactă). Nu mai există alte medii în `main.cpp` și celelalte `.h`.

💡 Cât dă `3 / 2` în C++, și cât dă `(double)3 / 2`?

B17 🔴 REDESCHIS: `algoritmi.h` tot inclus; compilează din nou doar pentru că ai editat materialul, care acum sortează greșit `algoritmi.h:168` · `cautare_binara.h:4` · `minim_maxim.h:4` · `sortare_bule.h:4` · `sortare_selectie.h:4`

CUM E ACUM: cele 4 headere includ încă `algoritmi.h`. În `d47270f` ai mutat `int aux = v[i];` sus în `algoritmi.h:168`, dar ordinea a rămas greșită: `aux = v[i]; v[i+1] = aux; v[i] = v[i+1]`. În plus, ai scris `frecventaCifraNumar` în `algoritmi.h` (`:384`). `cautare_binara.h:203-206` are tot 3 erori (`sol25`).

```
sortareIntervalDescrescator (algoritmi.h) pe 1 2 3 5 7 9, 0..5  ->  1 1 1 1 1 1
sortareMunte (algoritmi.h) pe 99 31 54 16 17 63, vârf 3
    ->  16 31 54 99 17 17     trebuia 16 31 54 99 63 17
cautare_binara.h singur:  :203:9: error: excess elements in scalar initializer
                        + 2 erori la :206
```

DE CE: Eroarea s-a dus, dar interschimbarea tot nu se face. Pe `v[i] = a`, `v[i+1] = b`: `aux = a`, `v[i+1] = a`, `v[i] = v[i+1]`, care e deja `a`. Rezultat: `a a`, iar `b` s-a pierdut. Așa că algoritmul de pe care înveți munte/vale (`sortareMunte` îl cheamă) acum pierde valori. Compilatorul a tăcut pentru că

`aux` era declarat înainte de folosire, dar nu verifică dacă ordinea are sens. Regula e să scoți include-ul, nu să repari materialul: un fișier de studiu stricat te învață greșit.

FIX: scoate `#include "algoritmi.h"` din cele 4 headere și scrie tu, sus în fiecare, funcțiile care lipsesc; `algoritmi.h` înapoi la versiunea din `c263119` (nu se modifică), iar `frecventaCifraNumar` mută-l în headerul tău.

- același bug la `sortare-interval.h:70-71` : aceeași interschimbare în `sortareIntervalDescrescator` ; la `:70` `v[i + 1] = aux;` folosește `aux` neinițializat (`:61`), apoi la `:71` `int aux = v[i];` declară un al doilea `aux` care îl umbrește; `v[i+1]` se pierde și valoarea e nedefinită (UB). Nu e inclus nicăieri azi, dar e aceeași greșeală.
- `algoritmi.h:168-171` : se repară tot blocul (ordinea e greșită la `:169` și `:171`), nu doar o linie
- `algoritmi.h:174-185` : `sortareMunte` și `sortareVale` apelează `sortareIntervalDescrescator` din `:163` , deci rezultatul greșit se propagă; nu mai au nevoie de altă modificare după reparare
- `cautare_binara.h:203` și `:206` : `int v = {...}` în loc de `int v[] = {...}` ; `parcurgereInVector` în loc de `cautareInVector`
- `cautare_binara.h:99` , `:116` , `:199` : funcții non- `void` fără `return` pe toate căile (warning `-Wreturn-type`)

💡 Trasează pe hârtie o interschimbare între `5` și `3` : ce valoare se salvează prima?

B27 🔴 EXTINS: funcții copiate identic din `algoritmi.h` , continuă `interclasare.h:6` · `munte_vale.h:88`

CUM E ACUM: `interclasare.h:6-40` `interclasare` e identică cu `algoritmi.h:190-224` (`diff` gol, 35 de linii). `munte_vale.h:88-105` `sortareInterval` identică cu `algoritmi.h:140-157` ; `sortareIntervalDescrescator` (`:106`) și `sortareMunte` (`:125`) sunt copiate din versiunea originală. (`stergereElement` din `stergere.h:27` e identică, dar are trasarea ta deasupra și oricum are 5 linii, deci o accept.)

```
diff <(sed -n 190,224p algoritmi.h) <(sed -n 6,40p interclasare.h)   -> fără ieșire
diff <(sed -n 140,157p algoritmi.h) <(sed -n 88,105p munte_vale.h)   -> fără ieșire
```

DE CE: Fișa spune: citești, închizi fișierul, scrii din memorie. Interclasarea e chiar algoritmul secțiunii 7. Dacă o copiezi, 7.3–7.5 (care sunt variații ale ei) rămân fără bază: ca să le faci, trebuie să știi de ce sunt trei `while` -uri și ce face fiecare. Dovada că se poate: `stergereElement` are trasarea ta deasupra, și de-aia știi de ce e nevoie de `i--` în `sol1` .

FIX: închide `algoritmi.h` , trasează interclasarea pe hârtie pe `a: 1 4 7 9` , `b: 2 3 8` , apoi scrie-o din memorie.

- același bug la `munte_vale.h:6-10` `sortareMunte` (≡ `algoritmi.h:178-182`), `:12-16` `sortareVale` (≡ `:184-188`), `:88-105` `sortareInterval` (≡ `:140-157`), `:106-123`

`sortareIntervalDescrescator` (copie cu linii mutate față de `:159–176`), `:125–129` `sortareMunte` a doua oară în același fișier

- `sortare-interval.h:40–57` `sortareInterval` (≡ `algoritmi.h:140–157`), `:59–77` `sortareIntervalDescrescator` (copie cu linii mutate față de `:159–176`)
- `stergere.h:5–12` `afisare` (≡ `algoritmi.h:8–15`), `:27–34` `stergereElement` (≡ `:226–233`)
- `interclasare.h:6–40` (cazul de mai sus) · `:42–46` `afisare(const char*, int[], int)`: alt prototip, aceeași idee (dublură parțială)

💡 Dacă te blochezi, trasează algoritmul pe hârtie; la `stergereElement` ai făcut-o deja foarte bine.

🟡 Importante

- **M18** — 6.3 și 6.5 cer o funcție care „întoarce"; `undeVarf` (`munte_vale.h:76`) și `cateVarfuri` (`:140`) afișează cu `cout` și sunt `void`. La fel `eMunte` / `eVale`: ar trebui să fie `bool`.
- **M19** — 8.1, 8.3 și 8.4 sunt scrise direct în `sol1` / `sol3` / `sol4`, nu ca funcții cu parametri, cum cere fișa (`stergere.h:41`, `:102`, `:129`).
- **M20** — `frecventaVector` (`stergere.h:152`): `f[v[i]]` cu `f[100]` iese din vector la orice valoare `>= 100` sau negativă.
- **M21** — nume duplicate: `sol1` / `sol2` în `stergere.h`, `minim_maxim.h` și `interclasare.h`; `afisare` în `stergere.h`, `interclasare.h` și `algoritmi.h`. Două headere incluse împreună nu mai compilează.
- **M22** — `interclasare.h:52–66`: sortare prin inserție, `MAX` nedefinit, parametrul `const char* nume` nefolosit, `"--- 7.2 ---"`: nu sunt din materialul tău. Scrie 7.2 cu sortarea pe care o știi (bule/selecție) din headerele tale.
- **M23** — 7.1 n-are `sol`; 7.3, 7.4, 7.5 nescrise.
- **M1** — garda cu cratimă: `sortare-interval.h:1`, `while-ex.h:1`, `parcurgerea-vectoriilor.h:1`. NEATINS.
- **M14**, **M4**, **M17** — NEATINSE (fișierele sunt identice cu runda 4).
- **M16** — `interclasare.h`, `munte_vale.h`, `stergere.h` acum au cod ✅; `sortate_bule.h` și `fisa2.h` tot goale.

🟢 Cleanups

- **C24** — `sol4` (`stergere.h:131–132`): 11 valori, `d = 10`, deci `20` nu intră în calcul.
- **C25** — `main.cpp:6–15`: bloc comentat de cod mort.
- **C26** — `129eeb4` și `6541fd6` au mesajul „."; scrie ce ai făcut (ex. „8.1–8.5 stergere").
- **C18** — `bin/Debug/recapitulare.exe` și `obj/Debug/main.o` tot în git, recomise în `d47270f`.

- **C27** — `stergere.h:147` typo-uri în comentariu („fectorul”, „vectot”).

Ce e bine

- **8.1 și 8.5 corecte** (`5 2 5 7 5` -> `2 7` , `5 5 3 7 7 1` -> `3 1`). `stergereAparitiiElementVector` cu `v[k++] = v[i]` e o idee foarte bună: ștergi într-o singură trecere, fără să muți tot vectorul de fiecare dată.
- **Logica de la munte/vale** e corectă pe toate exemplele 6.1–6.5 (verificată pe o copie reparată), inclusiv cazurile-capcană `1 4 7` și `3 3 5 2` .
- **Trasarea lui** `stergereElement` (`stergere.h:14-24`) și a frecvenței pe cifre; `i--` l-ai pus singur în 3 din 4 locuri.

Before / After (🔴)

	Before	After
B28	`int x[MAX], y[MAX];` și `sortareInterval` chemată la `:8` , definită la `:88`	`int x[100], y[100];` și `sortareInterval` definită înaintea lui `sortareMunte` ; un singur `sortareMunte`
B29	`stergereElement(v, d, i);`	`stergereElement(v, d, i); i--;`
B30	`sortare(v, d);` apoi `if (v[i] == v[i+1])` pe `i < d`	fără sortare: `for i<d: for j=i+1; j<d; j++: if (v[j]==v[i]) { stergereElement(v,d,j); j--; }`
B31	`return s / d;` și `int ma = mediaAritmetica(v, d);`	`return (double)s / d;` și `double ma = mediaAritmetica(v, d);`
B17	`#include "algoritmi.h"` în 4 headere; `aux = v[i]; v[i+1] = aux; v[i] = v[i+1];`	include scos, funcțiile scrise de tine; `aux = v[i]; v[i] = v[i+1]; v[i+1] = aux;`
B27	`interclasare` copiată din `algoritmi.h:190-224`	scrisă din memorie, după trasarea pe `a: 1 4 7 9` , `b: 2 3 8`

Q&A — răspunde în scris, înainte de runda 6

1. Pe `10 14 10 15`: ce se află pe poziția 1 imediat după ce ștergi `14` de pe poziția 1, și ce poziție verifică `for`-ul la pasul următor fără `i--`?

2. Pe `5 2 5 7 2 2 9`: de ce dispare `9`? Scrie ce e în `v[d-1]` și ce e în `v[d]` înainte de ultima comparație.

3. Cât dă `3 / 2` în C++, și cât dă `(double)3 / 2`? De ce nu ajută faptul că funcția `mediaAritmetica` întoarce `double`?

(Răspunsurile din runda 4 nu au apărut în repo; rămân deschise: `16777216` din 5.3, căutarea binară pe vector neordonat, `pozMaxim` vs `v[pozMaxim]`.)

Runda 4 — 2026-10-15 — sha 6c66dca (condensată)

Poarta rundei 4: `main.cpp` compila; separat pica `cautare_binara.h` și, după editarea `algoritmi.h`, toate cele 4 headere dependente. Starea fiecărei constatări, verificată pe `129eeb4`:

#	Loc	Constatare	Stare pe `129eeb4`
B17	`algoritmi.h:168`, 4 headere	`algoritmi.h` inclus; interschimbare greșită	🔴 REDESCHIS: compilează prin editarea materialului, interschimbarea rămâne greșită (vezi runda 5)
B18	`sortare-interval.h:59`	`sortareIntervalDescrescator` pe `1 2 3 5 7 9` -> `1 1 1 1 1 1`	🔴 NEATINS
B19	`cautare_binara.h`, `minim_maxim.h`, `sortare_bule.h`, `sortare_selectie.h`	funcții `int` fără `return`	🔴 NEATINS (fișierele identice cu `6c66dca`; `cautare_binara.h` tot necompilabil separat)
B20	`cautare_binara.h:142`, `:145`	`<` / `>` în loc de `<=` / `>=`	🔴 NEATINS (`cautare_binara.h` identic)

#	Loc	Constatare	Stare pe `129eeb4`
B21	`cautare_binara.h:48`, `:57`	căutare binară pe vector neordonat	🔴 NEATINS (`sol25`)
B22	`minim_maxim.h:64-69`	`aDouaValoare` compară valoare cu poziție	🔴 NEATINS: `9 3 8 1` -> `1` (trebuia 8); `3 3 8 1` -> 3, tot din întâmplare
B23	`sortare_bule.h:67`	`dejaOrdonat` sortează originalul	🔴 NEATINS: `dejaOrdonat(1 7 3)` -> 0, vectorul după apel `1 3 7`
B24	`sortare_selectie.h:26-35`	`ordonareDupaUc` afișează ultimele cifre	🔴 NEATINS: `23 41 15 32` -> `1 2 3 5`
B25	`sortare_selectie.h:49`	`v[i - 1]` la `i = 0`	🔴 NEATINS (identic)
B26	`sortare-interval.h:133`	`sortareTreiParti` pornește de la 1	🔴 NEATINS: `9 7 5 3 1 2` -> `9 7 3 5 1 2`
B27	`cautare_binara.h:108`, `sortare-interval.h:40`	funcții copiate din `algoritmi.h`	🔴 EXTINS: copiere și în `interclasare.h` și `munte_vale.h` (vezi runda 5)
B15	`parcurgerea-vectoriilor.h:183`	`produsCifre(234)` -> `4`	🔴 NEATINS
B16	`parcurgerea-vectoriilor.h:223`	„prime între ele" = amândouă prime	🔴 NEATINS (identic)
M1	`sortare-interval.h:1`, `while-ex.h:1`, `parcurgerea-vectoriilor.h:1`	garda cu cratimă	🟡 NEATINS
M14	—	`mediana`, `kValoare`, `primeleK` sortează vectorul primit	🟡 NEATINS
M4	`while-ex.h:345`	`cmmmc(50000, 70000)` -> `-79496`	🟡 NEATINS

#	Loc	Constatare	Stare pe `129eeb4`
M17	—	`cifraMaxima(0)` -> `-1`	🟡 NEATINS
M16	`interclasare.h`, `munte_vale.h`, `stergere.h`, `sortate_bule.h`, `fisa2.h`	fișiere goale comise	🟡 ✅ primele 3 au cod · `sortate_bule.h` și `fisa2.h` tot goale
C18	`bin/`, `obj/`	binare în git	🟢 NEATINS, recomise în `d47270f`
C20–C23	—	`afisareDistinceDescrescatoare`, variabile nefolosite, `primeleK` cu `k > dim`, mesaje de commit	🟢 nerevizuite în runda 5 (C23 continuă ca C26)
Restanțe r1	`while-ex.h`	B3, B4, B6, M2, M3, M6	nerevizuite în runda 5
Q&A	—	cele 3 întrebări ale rundei 4	niciun răspuns în repo

Runda 3 — 2026-10-01 — sha ee7010e (condensată, nelivrată la timp)

Generată pe 2026-10-01, n-a ajuns la tine. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B15	`parcurgerea:183`	`produsCifre`: `suma * n % 10` → mereu o cifră	🔴 NEATINS (vezi runda 4)
B16	`parcurgerea:223`	„prime între ele" verificat cu `isPrim`	🔴 NEATINS (vezi runda 4)
B9	`parcurgerea:55-57`	`isPrim(i)` în loc de `isPrim(v[i])`	✅ ÎNCHIS
M10	`parcurgerea:109`	`afisareRasturnat` testează doar capetele	✅ ai ales palindromul: `punctul_c` cu `ePalindrom`

#	Loc	Constatare	Stare pe `6c66dca`
M11	`parcurgerea:156`	`sol1d` nu poate afișa ramura adevărată	🟡 NEATINS
M12	`parcurgerea:20`, `:253`	Cod comentat care nu compilează	🟡 `punctul_c` rescris ✅; `sol1` și `punctul_g` tot comentate
M7	—	`isPrim` ≡ `estePrim`, `CifraMinima` ≡ `cifraMinima`	🟡 NEATINS
C14	`parcurgerea:107`, `:207`	Variabile nefolosite	🟢 NEATINS
C15	`parcurgerea:168`	`?:` cu `<<`	🟢 NEATINS
C16	`parcurgerea:142`	`divizoriProprii` întoarce un număr	🟢 NEATINS
C17	—	`sol12`	✅ redenumit `sol2`
C18	`bin/`, `obj/`	Binare în git	🟢 NEATINS

Runda 2 — 2026-10-01 — sha fdda59a (condensată)

Textul integral e în PDF-ul rundei 2 (commit `b3b1bf6`).

#	Constatare	Stare pe `6c66dca`
B8	Nu compila, 7 erori	✅ ÎNCHIS (runda 3)
B9	`contorPrimeVector` număra indicii	✅ ÎNCHIS
B10	Indexare de la 1 vs de la 0	✅ ÎNCHIS
B11	Palindromul testa doar capetele	✅ ÎNCHIS
B12	`punctul_g` — buclă infinită	✅ ÎNCHIS
B13	`if` în afara buclei	✅ ÎNCHIS

#	Constatare	Stare pe `6c66dca`
B14	Funcție `int` fără `return`	✅ `afisareaElemVect` e `void` · 🔴 tiparul revine în runda 4 (B19)
M7	Funcții rescrise, existente deja	🟡 NEATINS · copiere în runda 4 (B27)
M8	`int c[10]` cu indexare de la 1	✅ codul a fost rescris
M9	`punctul_g` folosește `nr` nesetat	🟡 cod comentat
C7	`CIfraMinima`	✅ `CifraMinima`
C8	`//todo:` învechit	✅
C9	`dim` declarat, literal pasat	🟢 NEATINS (`parcurgerea:69`)
C10, C11	`main.cpp` vechi	✅ `main.cpp` rescris
C12	Binare în git	🟢 NEATINS
C13	Mesaje de commit	🟢 NEATINS

Runda 1 — 2026-09-24 — sha 2734bb4 (condensată)

Livrată pe 2026-10-01, odată cu runda 2. Starea fiecărei constatări, verificată prin rulare pe `6c66dca`:

#	Loc	Constatare	Stare pe `6c66dca`
B1	`main.cpp:1`	`while-ex.h` nu ajungea la compilator	✅ compilat separat de mine: 0 erori
B2	`while-ex.h:72` , `:429` , `:352`	Cifra scoasă înainte de `while`	✅ ÎNCHIS în toate cele 3 locuri
B3	`while-ex.h:126` , `:171`	Două bucle cu `n` nemodificat → nu se termină	🔴 NEATINS
B4	`while-ex.h:124` , `:169`	Funcții `int` fără `return`	🔴 2 rămase · vezi B19
B5	`while-ex.h:303`	`cmmdc(0, 0)` → împărțire la zero	✅ ÎNCHIS

#	Loc	Constatare	Stare pe `6c66dca`
B6	`while-ex.h:452`	`eliminareCifra` scoate toate aparițiile	🔴 NEATINS
B7	`while-ex.h:370`	`celMaiMicNrCuSumaCifrelor(20)` → 99	✅ ÎNCHIS (funcția nouă)
M1	gărzile `.h:1`	Cratimă în numele macro-ului	🔴 NEATINS + `sortare-interval.h`
M2	`while-ex.h:154`, `:201`	`% p` în loc de `% 10`	🔴 NEATINS
M3	`while-ex.h:479`, `:554`	`pow` fără `<cmath>`	🔴 NEATINS
M4	`while-ex.h:345`	`a * b` → depășire	🔴 reprodus: `cmmmc(50000,70000)` → `-79496`
M5	`fisa2.h`	Fișier gol	🔴 NEATINS · vezi M16
M6	toate `.h`	Definiții în `.h`	🟡 NEATINS
C1–C3	—	`dim`/literal, `int n` nefolosit, `isZero`	🟢 NEATINSE
C4	`bin/`, `obj/`	Binare în git	🟢 NEATINS
C5	`while-ex.h:8`, `main.cpp:3`	`//todo:` învechite	🟢 `main.cpp` ✅, `while-ex.h:8` rămas
C6	—	Mesaje de commit	🟢 NEATINS